ACTIVIDAD PRACTICA

Diseño arquitectural del sistema con IA

Guía de trabajo para alumnos

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto.

Empresa asignada	Hotel Costa Azul
Integrantes	Ivan Francisco Padron Macias – 2150830 María Ximena Rodarte Cuevas – 2154111 Angel Haziel Bernal Garza - 2156444
Fecha de entrega	12/10/2026

Cisco Confidential

1 Proposito de la actividad

El objetivo es organizar las reglas y responsabilidades del sistema antes de programarlo por completo. Al terminar, el equipo deberá demostrar cómo pasó de los requerimientos de la Unidad 2 a una arquitectura que separa el dominio, la aplicación y la infraestructura.

La actividad no consiste en pedir a una herramienta de IA que produzca una solución terminada. El trabajo consiste en plantear decisiones, solicitar alternativas, revisar las respuestas y documentar por qué una propuesta sí corresponde o no al negocio analizado.

Resultados de aprendizaje

- Comparar Transaction Script, Table Module y Domain Model con base en la complejidad real del caso.
- Modelar el dominio mediante lenguaje ubicuo, entidades, Value Objects, Aggregates y Domain Events.
- Diseñar una Service Layer que coordine casos de uso sin absorber las reglas del dominio.
- Separar la persistencia mediante Repository y definir el alcance transaccional con Unit of Work.
- Distinguir Commands y Queries en una propuesta básica de CQRS.
- Evaluar la arquitectura con los principios SOLID y registrar las decisiones en ADR.
- Usar IA como copiloto y conservar evidencia de la revisión humana.

2 Continuidad con la Unidad 2

Trabajarán con la misma empresa y no comenzarán un caso nuevo. Antes de diseñar la arquitectura, reúnan los productos elaborados en la Unidad 2 y asegúrense de que todos los integrantes comprenden el proceso principal del negocio.

Insumo de la Unidad 2	Como se utilizara en la Unidad 3
BRD	Delimitar alcance, actores, necesidades, restricciones, supuestos y criterios de exito.
Historias de usuario y Gherkin	Identificar casos de uso, condiciones de negocio y comportamientos esperados.
BPMN	Reconocer la secuencia, decisiones, participantes, excepciones y puntos de coordinacion.
Glosario y modelo de dominio	Mantener un lenguaje ubicuo y localizar conceptos con identidad, valor o ciclo de vida.
Matriz de trazabilidad	Relacionar cada decision arquitectural con una necesidad, historia, regla o escenario.
Repositorio de GitHub	Conservar versiones, diagramas, documentos, prompts y codigo generado o revisado.

Regla de continuidad

Si el equipo detecta una contradicción o un dato faltante en la Unidad 2, deberá registrarlo como hallazgo. Puede ajustar el análisis anterior, pero debe explicar el cambio y actualizar la trazabilidad. No se permite inventar nuevas reglas solo para acomodar una arquitectura.

Cisco Confidential

3 Empresas de trabajo

Cada equipo continuará con la empresa que trabajó en la Unidad 2. La siguiente tabla recuerda el contexto general de los nueve casos. Las preguntas sirven para enfocar la revisión; no indican qué patrón, clases o componentes deben elegir.

Empresa	Contexto	Pregunta de analisis
Mercado Norte	Alimentos, perecederos y promociones	¿Que reglas cambian por producto, sucursal, fecha o existencia?
TecnoNova Retail	Electronica, numeros de serie y garantias	¿Que decisiones requieren conservar identidad y ciclo de vida?
Moda Urbana	Ropa, tallas, colores y temporadas	¿Donde aparecen variantes, reservas, cambios y devoluciones?
HogarMax	Hogar, unidades voluminosas y ventas a credito	¿Que operaciones coordinan inventario, credito y entrega?
SaludPlus Farmacias	Medicamentos, lotes y caducidad	¿Que invariantes deben cumplirse antes de autorizar una operacion?
SportZone	Articulos deportivos y comisiones	¿Que reglas pertenecen a la venta y cuales a procesos posteriores?
Pet and Co	Productos, servicios y suscripciones	¿Que comportamientos dependen del plan, vigencia o tipo de servicio?
Casa and Estilo	Muebles, anticipos y entregas	¿Que estados y eventos cambian durante el cumplimiento de un pedido?
LibroMundo	Libros, ISBN y preventas	¿Que cambia entre una venta normal, una preventa y una cancelacion?

4 Reglas generales

- Mantengan el alcance aprobado en el BRD y seleccionen un proceso principal para el diseño detallado.
- Todas las decisiones deben relacionarse con requerimientos, reglas, historias o escenarios existentes.
- No se evaluará la cantidad de clases. Se evaluará si cada responsabilidad está ubicada y justificada correctamente.
- Los nombres usados en diagramas, código y documentos deben coincidir con el glosario del negocio.
- El código puede ser parcial o boilerplate. Debe permitir revisar la organización y las responsabilidades propuestas.
- Conserven las iteraciones realizadas con IA. No entreguen una respuesta generada sin revisión.

5 Ruta de trabajo

Realicen las etapas en orden. Una etapa puede obligarlos a corregir una decisión anterior; registren el cambio en lugar de ocultarlo.

Cisco Confidential

Etapa	Trabajo	Resultado
1	Preparar el caso	Revisar U2, delimitar proceso y seleccionar casos de uso.
2	Elegir el patron	Comparar alternativas y justificar la opcion elegida.
3	Construir el mapa de dominio	Definir lenguaje, responsabilidades, limites y eventos.
4	Diseñar la Service Layer	Organizar la coordinacion de los casos de uso.
5	Separar persistencia y transacciones	Proponer Repository y Unit of Work.
6	Separar comandos y consultas	Aplicar CQRS basico donde tenga sentido.
7	Revisar con SOLID	Detectar acoplamiento y responsabilidades incorrectas.
8	Representar la arquitectura	Crear UML, secuencias y ADR.
9	Validar y entregar	Trazar, probar, depurar y organizar evidencias.

Etapa 1 Preparar el caso

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Consulta y Cálculo de Tarifas con Disponibilidad	Reservaciones	US-02, US-03	El sistema devuelve la lista de habitaciones disponibles y cotiza correctamente o indica error de fechas.
CU-02	Confirmación de Reserva y Prevención de Overbooking	Reservaciones	US-04, US-25	Inventario sincronizado sin duplicidades o rechazo por falta de disponibilidad.
CU-03	Proceso de Check-in con Validación de Datos y Factura	Recepción	US-07	Datos guardados, habitación en estado "Ocupada" o bloqueo por falta de datos fiscales.
CU-04	Registro de Consumos y Cierre de Folio (Check-out)	Recepción	US-10, US-13	Trazabilidad completa de cargos o prohibición de cierre con cuentas pendientes.

Cisco Confidential

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-05	Gestión de Excepciones por Overbooking (*Walk*)	Recepción	US-23	Reserva marcada como reubicada con datos completos o rechazo por datos faltantes.

Etapa 2 Elegir el patrón de lógica de negocio

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Baja. Lógica procedural lineal; propensa a duplicar lógica de validación de tarifas y horarios en múltiples flujos.	Media. Centrada en operaciones sobre conjuntos de datos tabulares, útil para reportes masivos pero limitada para lógica relacional fina.	**Alta y Encapsulada.** Modela reglas complejas de negocio, cálculo de tarifas dinámicas y restricciones cruzadas directamente en los objetos.	BRD: Reglas complejas y acopladas (RN-01 a RN-13) sobre tarifas, horarios, garantías y roles.
Cambios esperados	Alta fricción y riesgo de errores. Un cambio en políticas (ej. penalización por cancelación) obliga a modificar código en múltiples scripts.	Moderada. Permite modificaciones eficientes si están estrictamente ligadas a estructuras de bases de datos tabulares.	**Baja fricción.** Los cambios en políticas de negocio se localizan en la entidad o Value Object correspondiente, minimizando efectos colaterales.	BRD: Necesidad de automatizar la operación para adaptarse dinámicamente a la disponibilidad de canales y políticas del hotel.
Interacción entre conceptos	Débil. Los datos y el comportamiento están separados; se procesan registros planos sin un grafo de relaciones del mundo real.	Basada en tablas. Ideal para consultas masivas de filas y columnas, pero ineficiente para modelar comportamientos de ciclo de vida de entidades.	**Rica y Colaborativa.** Representa fielmente el mundo real mediante un grafo de objetos (Reserva, Folio, Habitacion, Huésped) que interactúan entre sí.	BPMN y Glosario: El ciclo de vida del huésped cruza múltiples departamentos (Reservaciones, Recepción, Limpieza, Administración).
Estados e invariantes	Inexistente de forma nativa. Las entidades son estructuras anémicas; es fácil dejar bases de datos en estados inconsistentes por omisiones en los scripts.	Moderado a nivel de conjunto de datos, pero incapaz de gobernar reglas de ciclo de vida complejas por entidad individual.	**Estricta y Robusta.** Las entidades protegen sus propios invariantes; un objeto rechaza operaciones si no cumple con las condiciones del negocio.	BRD y User Stories: Control de estados críticos (Habitaciones *Fuera de Servicio*, folios abiertos, bloqueo de 10+ habitaciones con anticipo).

Cisco Confidential

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Criterio adicional	Las excepciones por permisos (ej. límites de ajuste de USD 50) generan una maraña de condicionales dispersos en los procedimientos.	No maneja adecuadamente flujos excepcionales de autorización de roles gerenciales ni transacciones de cancelación con penalización.	**Delegado y Elegante.** Las reglas de autorización y excepciones forman parte natural del comportamiento del modelo de dominio.	User Stories (US-06, US-18, US-19, US-23): Manejo estricto de excepciones, overbooking, no-shows y límites de autorización gerencial.

Justificación Arquitectónica: Elección de Domain Model para el Hotel Costa Azul

La elección de **Domain Model** para el Hotel Costa Azul se propone como una forma de organizar la lógica interna del Sistema de Gestión Hotelera (PMS, por sus siglas en inglés). El PMS permite administrar las reservaciones, las habitaciones, el registro de entrada y salida de los huéspedes, así como los cargos y pagos. Por su parte, Domain Model permite organizar las reglas y operaciones de estas funciones para que el sistema trabaje de manera ordenada, reduzca errores y facilite la adaptación a futuros cambios en las políticas del hotel.

1. Por qué se descartan los patrones alternativos

- **Por qué se descarta *Transaction Script*:** Este patrón organiza la lógica del sistema mediante instrucciones que se ejecutan en orden para realizar acciones específicas, como registrar un *check-in* o calcular una penalización. Si se utilizara en el Hotel Costa Azul, las reglas del negocio podrían repetirse en diferentes partes del sistema. Por ejemplo, las tarifas según la temporada, los incrementos de fin de semana y las penalizaciones por *no-show* tendrían que manejarse en varios procedimientos. Si alguna política del hotel cambiara, sería necesario modificar diferentes partes del código, aumentando el riesgo de errores en los cobros y en las operaciones.
- **Por qué se descarta *Table Module*:** Aunque el hotel necesita generar reportes sobre la ocupación y los ingresos, su sistema de gestión (PMS) también debe controlar las operaciones diarias y los cambios en el estado de las habitaciones. Una habitación no es solamente un registro en una tabla, ya que puede estar limpia, sucia, inspeccionada o fuera de servicio, y cada cambio debe cumplir ciertas reglas. Este patrón organiza la lógica a partir de las tablas de datos, pero resulta menos adecuado para manejar operaciones complejas, como transferir cargos entre cuentas de huéspedes o administrar reservaciones de grupos que requieren anticipos.

2. Problemas concretos que resuelve *Domain Model*

La elección de Domain Model responde directamente a los problemas operativos identificados en el análisis *AS-IS* del hotel, ya que permite organizar las reglas del negocio y controlar las operaciones de acuerdo con las necesidades del establecimiento:

- **Protección de reglas y estados:** Este modelo permite que cada elemento del sistema tenga sus propias reglas. Por ejemplo, una habitación marcada como fuera de servicio no podrá asignarse a un huésped, y una cuenta de estancia no podrá cerrarse durante el *check-out* si todavía existen saldos pendientes. De esta manera, se reducen los errores que podrían surgir al realizar estas operaciones manualmente.

Cisco Confidential

- **Manejo de reglas de negocio y excepciones:** El PMS debe aplicar diferentes políticas, como cobrar la primera noche en caso de *no-show* cuando existe una garantía válida, así como respetar las condiciones de cancelación con 48 horas de anticipación. Con Domain Model, estas reglas se organizan dentro de los elementos correspondientes (como Reserva y Folio), lo que facilita su comprensión, mantenimiento y modificación cuando cambien las políticas del hotel.
- **Control de autorizaciones por roles:** El hotel establece que solamente el personal autorizado, como la Gerencia de Operaciones o Recepción, puede realizar determinadas acciones, como eliminar penalizaciones, modificar tarifas confirmadas o habilitar habitaciones que estaban fuera de servicio. Este modelo permite establecer controles para verificar los permisos del usuario antes de aceptar estos cambios, ayudando a mantener la seguridad y el cumplimiento de las políticas internas.

3. Costos introducidos por la decisión

- **Curva de aprendizaje y complejidad de diseño:** En comparación con un enfoque más sencillo, Domain Model requiere más tiempo de análisis y planificación para definir los elementos del sistema, sus responsabilidades y la forma en que se relacionan entre sí. Por ello, el equipo debe comprender bien las operaciones y reglas del hotel antes de comenzar el desarrollo.
- **Mayor trabajo inicial de implementación:** Este patrón requiere mecanismos adicionales para organizar, guardar y recuperar la información del sistema, como las reservaciones, habitaciones y cuentas de los huéspedes. Esto aumenta la cantidad de trabajo y código necesarios al inicio del proyecto. Sin embargo, este esfuerzo se compensa con una mayor facilidad para realizar cambios, corregir errores y adaptar el sistema a las futuras necesidades del Hotel Costa Azul.

Etapa 3 Construir el mapa de dominio

El mapa de dominio reúne en un solo diagrama los Aggregates del negocio (Reserva y EstanciaFolio, junto con las entidades de referencia como Habitación y Huésped), las entidades y Value Objects que viven dentro de cada uno, las referencias entre Aggregates (solo por identificador), los servicios del dominio (TariffCalculationService y WalkService), las reglas que protege cada Aggregate y sus Domain Events. Cubre todo el alcance del BRD para servir de base al proyecto final: los elementos con borde punteado no participan en los cinco casos de uso de esta unidad y se construirán en la implementación.

Cisco Confidential

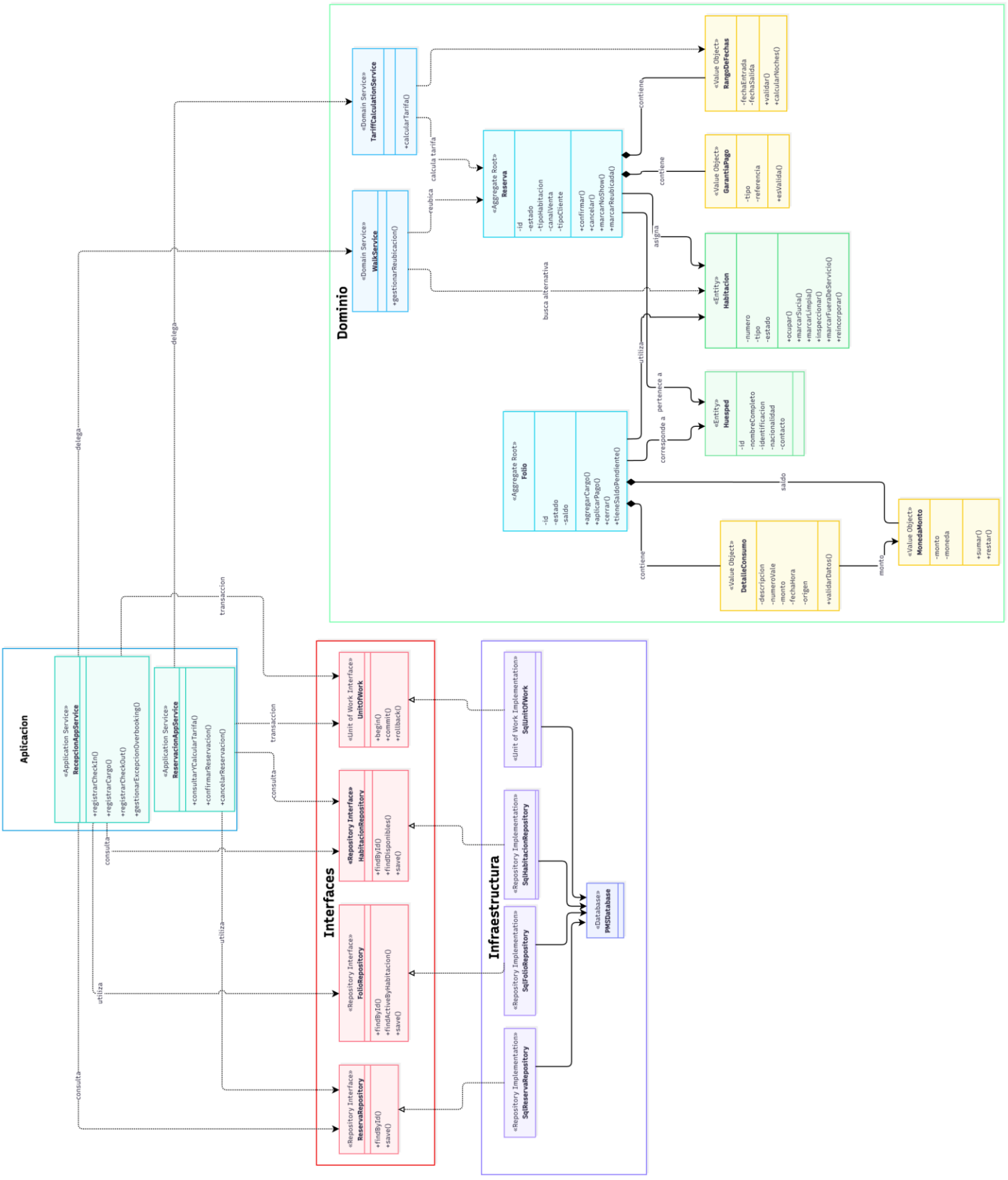


Figura 1: Mapa de dominio estratégico y táctico del PMS "Hotel Costa Azul", que integra los agregados principales (Reserva y EstanciaFolio), entidades, Value Objects, servicios de dominio y eventos que gobiernan el alcance del negocio.

Cisco Confidential

- Lenguaje ubicuo

Término	Definición en el Lenguaje Ubicuo del Equipo	Tipo / Nombre del Diseño
Aggregate / Aggregate Root	Grupo de objetos del dominio tratados como una unidad de consistencia transaccional. La raíz es el único punto de entrada para leer o modificar el grupo; nada externo puede alterar un miembro interno sin pasar por ella.	Diseño Táctico DDD (Reserva, Folio)
Value Object	Concepto del dominio definido exclusivamente por sus atributos; carece de identidad propia. Dos instancias con los mismos valores son idénticas e inmutables.	Diseño Táctico DDD (RangoDeFechas, MonedaMonto, DetalleConsumo, GarantíaPago)
Domain Event	Hecho del negocio que ya ocurrió y es relevante para otros componentes del sistema. Se nombra estrictamente en pasado y es inmutable.	Diseño Táctico DDD (ReservaConfirmada, CheckInRegistrado)
Application Service	Orquestador de un caso de uso. Recibe la solicitud, obtiene agregados vía repositorio, invoca el comportamiento del dominio y solicita persistencia. No contiene reglas de negocio propias.	Capa de Aplicación (ReservacionAppService, RecepcionAppService)
Repository	Abstracción del dominio que oculta los detalles de persistencia. Sus métodos hablan exclusivamente en términos del negocio (findDisponibles, findActiveByHabitacion).	Capa de Infraestructura (ReservaRepository, FolioRepository, HabitacionRepository)
Unit of Work	Patrón que agrupa múltiples operaciones de modificación en una sola transacción atómica, garantizando que todos los cambios se persistan juntos o se reviertan.	Capa de Infraestructura / Persistencia
Bounded Context	Límite semántico explícito dentro del cual un modelo de dominio es válido y coherente (ej. *Reservaciones* y *Estancia/Folio*), donde un mismo concepto puede tener significados distintos.	Arquitectura Estratégica DDD
Invariante	Regla de negocio crítica que debe cumplirse siempre dentro de los límites de un agregado. El agregado la protege rechazando operaciones que la violarían (ej. el folio no puede cerrar con saldo pendiente).	Diseño Táctico DDD (Reglas de Negocio)
BAR (Best Available Rate)	La tarifa pública base y vigente para una habitación en un momento determinado, sin restricciones especiales. Sirve como punto de partida para el cálculo de precios.	Dominio de Negocio / Value Object
Folio	Cuenta contable abierta y activa durante la estancia del huésped. Mantiene su identidad ligada a la habitación y registra la trazabilidad acumulada de todos los cargos y pagos.	Entidad / Raíz de Agregado (EstanciaFolio)
No-show	Situación operativa excepcional en la que un huésped con reserva confirmada no se presenta en la fecha prevista ni cancela previamente dentro del plazo.	Evento / Regla de Negocio (RN-05)
Habitación Fuera de Servicio (OOO)	Condición operativa en la que una unidad de hospedaje no puede ser comercializada temporalmente debido a mantenimiento o daños. Requiere autorización gerencial.	Estado Operativo de la Entidad Habitación
Walk (Reubicación)	Procedimiento excepcional en el cual, ante overbooking al momento del check-in, el hotel reubica al huésped en un establecimiento aliado cubriendo diferencias y cortesías.	Domain Service (WalkService)
Estado de Habitación	Máquina de estados estrictos que rige el ciclo físico: Disponible ➔ Ocupada ➔ Sucia ➔ Limpiada ➔ Inspeccionada ➔ Disponible.	Regla de Ciclo de Vida de Entidad

- Elementos usados

- Glosario actualizado con los términos nuevos o corregidos.

Término	Definición en el lenguaje ubicuo del equipo	Estado
Aggregate / Aggregate Root	Grupo de objetos del dominio tratados como una unidad de consistencia. La raíz (root) es el único punto de entrada para leer o modificar el grupo; nada externo puede modificar un miembro interno sin pasar por ella.	Nuevo
Value Object	Concepto del dominio definido exclusivamente por sus atributos; no tiene identidad propia. Dos instancias con los mismos valores son idénticas. Ejemplos: RangoDeFechas, MonedaMonto, DetalleConsumo.	Nuevo
Domain Event	Hecho del negocio que ya ocurrió y es relevante para uno o más componentes del sistema. Se nombra en pasado (ReservaConfirmada, CheckInRegistrado). No se puede deshacer; solo se puede compensar.	Nuevo
Application Service	Orquestador de un caso de uso. Recibe la solicitud, obtiene agregados via repositorio, invoca el comportamiento del dominio y solicita la persistencia. No contiene reglas de negocio propias.	Nuevo
Repository	Interfaz del dominio que abstrae el acceso a la persistencia. Sus métodos hablan en términos del negocio (findDisponibles, findActiveByHabitacion), no	Nuevo

Cisco Confidential

	en SQL ni en tablas.	
Unit of Work	Patrón que agrupa las operaciones de un caso de uso en una sola transacción atómica. Garantiza que todos los cambios se persisten juntos o ninguno se persiste.	Nuevo
Bounded Context	Límite explícito dentro del cual un modelo de dominio es válido y coherente. En el caso: Reservaciones y Estancia/Folio. Un mismo término (Habitación) puede tener distinto significado en cada contexto.	Nuevo
Invariante	Condición de negocio que debe cumplirse siempre dentro de un aggregate. El aggregate la protege rechazando operaciones que la violarían. Ejemplo: el folio no puede cerrarse con saldo pendiente.	Nuevo
Command / Query	Separación CQRS: un Command modifica el estado del sistema y no retorna datos; una Query consulta el estado sin modificarlo. Facilita optimizar lectura y escritura de forma independiente.	Nuevo
RangoDeFechas	Value Object que encapsula la fecha de check-in y la de check-out. Su regla intrínseca garantiza que la salida sea estrictamente posterior a la entrada; cualquier instancia inválida es rechazada en construcción.	Nuevo
DetalleConsumo	Value Object que representa un cargo generado en un punto de venta (restaurante, spa). Contiene: número de habitación, nombre y firma del huésped, fecha/hora, origen, descripción, monto y folio del vale (RN-12).	Nuevo
GarantíaPago	Value Object que encapsula los datos del método de respaldo aportado al check-in o en la reserva. Su ausencia no impide registrar la reserva, pero genera una advertencia de riesgo (US-08).	Nuevo
MonedaMonto	Value Object que representa cualquier cantidad monetaria con su divisa (USD/MXN). Encapsula operaciones aritméticas seguras e impide mezclar monedas sin conversión explícita.	Nuevo
Folio	En U2 se definió como "registro de cargos". En U3 se precisa: es la raíz del aggregate EstanciaFolio; tiene identidad propia, ciclo de vida (Abierto → Cerrado) e invariantes propias (saldo debe ser cero al cerrar).	Corregida
Habitación	En U2 era un dato. En U3 se precisa: es una entidad con identidad y ciclo de vida operativo (Disponible → Ocupada → Sucia → Limpia → Inspeccionada; o Fuera de Servicio). Su estado es clave para dos bounded contexts.	Corregida
Estado de habitación	En U2: "condición operativa en un momento dado". En U3 se precisa la máquina de estados completa: Disponible → (check-in) → Ocupada → (check-out) → Sucia → (limpieza) → Limpia → (inspección) → Disponible. Fuera de Servicio es un estado orthogonal que requiere autorización gerencial para salir.	Corregida

- Ficha de cada Aggregate que indique su raíz, responsabilidades, invariantes y eventos posibles.

Aggregate	Raíz	Responsabilidades	Invariantes	Eventos posibles
Reserva	**Reserva (Aggregate Root)**	1. Gestionar el ciclo de vida de la reserva (Confirmada → Cancelada → No-Show → Reubicada) 2. Verificar disponibilidad y bloquear la habitación para el rango de fechas antes de confirmar 3. Calcular y registrar la tarifa según temporada, canal de venta y tipo de cliente (RN-13) 4. Aplicar políticas de cancelación y no-show de forma automática (RN-04, RN-05) 5. Registrar la garantía de pago y alertar si falta 6. Gestionar bloqueo para reservas	1. No pueden existir dos reservas confirmadas para la misma habitación en las mismas fechas (RN-06) 2. La fecha de salida debe ser posterior a la entrada 3. Cancelación con <48 h genera cargo de primera noche como penalización (RN-04) 4. No-show después de 23:59 activa cobro si existe tarjeta de garantía (RN-05) 5. Reserva grupal requiere depósito del 30% antes de confirmarse (RN-07) 6. La tarifa confirmada no puede modificarse sin autorización gerencial	• ReservaConfirmada • ReservaCanceladaSinCargo • ReservaCanceladaConPenalización • ReservaNoShow • ReservaReubicada (overbooking → hotel aliado) • ReservaGrupalConfirmada

Cisco Confidential

		grupales (≥10 habitaciones, depósito 30%, RN-07)	(RN-08)	
EstanciaFolio	**Folio (Aggregate Root)**	1. Controlar la estancia activa desde el check-in hasta el check-out del huésped 2. Registrar y validar cargos al folio desde todos los puntos de venta del hotel (RN-12) 3. Gestionar cambios de habitación durante la estancia actualizando el estado de ambas 4. Verificar saldo al cierre e impedir el check-out con saldo pendiente (US-13) 5. Aplicar cargos de late check-out según hora de salida (RN-03) 6. Mantener trazabilidad de cambios con el usuario que los autorizó (RN-08, RN-10)	1. Ningún cargo puede agregarse sin los datos obligatorios del vale de consumo (RN-12) 2. El folio no puede cerrarse con saldo distinto de cero (US-13) 3. Ajuste al folio >USD 50 requiere autorización gerencial previa (RN-10) 4. El estado de la habitación debe actualizarse a 'Sucia' sincrónicamente al check-out 5. Cambio de habitación solo posible si la habitación destino está disponible (US-11) 6. Solo existe un folio activo por habitación en un momento dado	• CheckInRegistrado • CargoAgregadoAlFolio • AjusteAplicadoAlFolio • HabitacionCambiada • CheckOutRegistrado • HabitacionFueraDeServicio (declarada / reincorporada)

- Explicación de las decisiones dudosas o descartadas.

Pregunta	Duda	Decisión
¿Debería Habitación ser su propio Aggregate?	Podría serlo debido a su identidad, ciclo de vida y reglas de estado (*Disponible*, *Sucia*, *Fuera de servicio*).	Se mantiene como entidad referenciada por ID. Sus cambios de estado ocurren siempre derivados de una Reserva (al confirmar) o de unFolio (al hacer check-out). No hay casos de uso en este alcance que modifiquen la habitación de forma aislada.
¿Debería WalkService vivir en el bounded context de Reservaciones?	El overbooking (*walk*) ocurre en el check-in cuando no hay habitación disponible, lo que involucra a la reserva.	Se ubicó en el bounded context de Estancia. La operación requiere gestionar la habitación física, abrir el folio en el hotel aliado y registrar cargos de cortesía (elementos de Estancia). La reserva original solo pasa a estado "Reubicada" mediante un evento.
¿Es Huésped un Aggregate Root o solo una Entidad?	Tiene identidad, datos de contacto, historial y podría proteger sus propias reglas fiscales.	Se modela solo como Entidad de apoyo (no como Aggregate Root). En los casos de uso actuales nunca se modifica de forma independiente a una Reserva o un Folio. Si en el futuro se implementa un CRM, se promoverá a su propio bounded context.
¿Debería TariffCalculationService ser parte del aggregate Reserva?	El cálculo de tarifas ocurre exclusivamente para una reserva, podría calcularse internamente.	Se mantiene como Domain Service independiente. Centraliza la lógica compleja (temporadas, canales, tipos de cliente según RN-13) para evitar que la entidad Reserva acumule dependencias externas, protegiendo el principio de Responsabilidad Única (S de SOLID).
¿Es la autorización gerencial parte del dominio o de la capa de aplicación?	Las reglas RN-08 y RN-10 exigen gerencia para descuentos o ajustes >USD 50, dudando si es regla de acceso o de dominio.	Se divide responsabilidades: La validación del *rol* (quién puede invocar) pertenece al servicio de aplicación; pero el *límite de USD 50* y el registro del folio son invariantes del dominio. El aggregate rechaza ajustes mayores si no va acompañado de una autorización válida, blindando la regla de negocio.

Cisco Confidential

Etapa 4 Diseñar la Service Layer

Servicio	Metodo o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
ReservacionAppService	consultarYCalcularTarifa	Rango de fechas, tipo de habitación, canal de venta y tipo de cliente.	HabitacionRepository , TariffCalculationService.	Lista de habitaciones disponibles y cotización estimada.	Cálculo de tarifas: TariffCalculationService determina la tarifa según la temporada y el día. La disponibilidad se consulta mediante el repositorio correspondiente.
ReservacionAppService	confirmarReservacion	Datos del huésped, rango de fechas, tipo de habitación, tarifa y garantía de pago.	ReservaRepository, HabitacionRepository, agregado Reserva, UnitOfWork y EventBus.	Confirmación con identificador de reserva y estado actualizado.	Confirmación y disponibilidad (RN-06): El agregado Reserva valida las condiciones de la reserva. Los mecanismos de disponibilidad y concurrencia verifican y protegen el inventario frente a solicitudes simultáneas para evitar el overbooking. El servicio coordina el guardado y la publicación del evento.
RecepcionAppService	registrarCheckIn	Identificador de reserva y datos de identidad del huésped.	ReservaRepository, FolioRepository, agregados Reserva y EstanciaFolio, UnitOfWork.	Registro de entrada y apertura del folio de estancia.	Condiciones de entrada: El dominio verifica que la reserva permita registrar la entrada. Se aplican las validaciones obligatorias de identidad. Los datos fiscales se exigen obligatoriamente cuando se solicita factura, y la asignación de la habitación respeta su estado operativo y disponibilidad actual.
RecepcionAppService	registrarCargo	Identificador del folio, concepto, importe y datos del consumo.	FolioRepository, agregado EstanciaFolio, UnitOfWork y POS.	Folio actualizado con el nuevo cargo.	Registro de cargos: El dominio valida los datos del cargo, el importe y las condiciones para agregar consumos al folio. El servicio coordina su registro y persistencia.
RecepcionAppService	registrarCheckOut	Identificador del folio y solicitud de cierre de estancia.	FolioRepository, agregado EstanciaFolio y UnitOfWork.	Confirmación del cierre del folio y de la estancia.	Cierre de estancia: La raíz del agregado Folio verifica las condiciones de cierre, incluidos los saldos pendientes conforme a las políticas del hotel. Si no se cumplen, rechaza la operación.
RecepcionAppService	gestionarExcepcionOverbooking	Identificador de reserva, motivo de reubicación y datos del hotel aliado.	ReservaRepository, HabitacionRepository, agregado Reserva, servicio de disponibilidad y UnitOfWork.	Registro de la reubicación y sus datos asociados.	Reubicación por overbooking (RN-06): El dominio valida los cambios permitidos en la reserva y las condiciones de la operación. El servicio coordina la búsqueda de alternativas y el registro de la reubicación conforme a las políticas y autorizaciones del hotel.

Cisco Confidential

Revisión obligatoria

Criterio de Evaluación de la Capa de Aplicación	Análisis y Respuesta en el PMS "Hotel Costa Azul"
¿El nombre del método expresa una acción del negocio?	**Sí.** Cada método refleja una intención clara y una acción del dominio hotelero dentro del PMS: consultarYCalcularTarifa, confirmarReservacion, registrarCheckIn, registrarCargo, registrarCheckOut y gestionarExcepcionOverbooking. Se evitan nombres técnicos genéricos (save, update), utilizando verbos orientados al negocio.
¿El método coordina o contiene decisiones que deberían estar en el dominio?	El servicio coordina el flujo: realiza validaciones de entrada, gestiona errores operativos y controla la secuencia de la operación. Sin embargo, **no se apropia de las reglas del negocio**: delega los cálculos, estados e invariantes directamente en los agregados (Reserva, EstanciaFolio) y en los Domain Services (TariffCalculationService).
¿El servicio depende de interfaces o de detalles concretos de base de datos?	**Depende de abstracciones diseñadas:** Los servicios de aplicación se estructuran para depender de abstracciones (como las interfaces previstas para los repositorios y UnitOfWork), evitando depender directamente de los detalles específicos de la base de datos o de la infraestructura.
¿La operación tiene un inicio y un final transaccional comprensible?	**Sí.** Los cambios incluidos en la transacción se ejecutan de manera atómica y se revierten cuando ocurre un fallo, de acuerdo con los mecanismos de persistencia implementados mediante el patrón Unit of Work.
¿Los errores del dominio pueden explicarse con términos del negocio?	**Sí.** Las excepciones y rechazos devuelven mensajes utilizando estrictamente el lenguaje ubicuo del hotel (por ejemplo: *"No hay disponibilidad de habitación para la fecha solicitada"*, *"El folio no puede cerrarse por saldos pendientes"* o *"El vale de consumo está incompleto"*), facilitando la comprensión operativa.

Repository	Necesidad del dominio	Metodos propuestos	Objeto que recupera o guarda
ReservaRepository	Gestionar el ciclo de vida de la reserva y proteger sus condiciones de negocio.	findById(id: UUID): Reserva save(reserva: Reserva): void	Recuperar y guardar el agregado Reserva (incluyendo sus Value Objects como RangoFechas y GarantiaPago).
HabitacionRepository	Consultar el inventario de habitaciones y verificar la disponibilidad real para un periodo y tipo determinados (RN-06).	findDisponibles(rangoFechas: RangoFechas, tipoHabitacion: TipoHabitacion): List<Habitacion>	Consultar las habitaciones disponibles y gestionar su estado operativo.
FolioRepository	Controlar la estancia activa del huésped, registrar consumos validados (RN-12) y verificar saldos pendientes antes del cierre (US-13).	findById(id: UUID): EstanciaFolio findActiveByHabitacion(habitacionId: String): EstanciaFolio save(folio: EstanciaFolio): void	Recuperar y guardar el agregado EstanciaFolio (con su raíz Folio y colecciones de DetalleConsumo).

Etapa 6 Separar Commands y Queries

Trabajo solicitado

- Seleccionen al menos dos operaciones que cambien el estado y descríbanlas como Commands.
- Seleccionen al menos dos necesidades de información y descríbanlas como Queries.
- Definan el Handler de cada operación y sus dependencias.
- Diseñen el DTO o View Model de lectura sin reutilizarlo automáticamente como modelo de escritura.
- Justifiquen si alguna parte del caso no necesita esta separación.

Cisco Confidential

Tipo	Nombre	Datos de entrada	Handler	Resultado	Justificacion
Command	ConfirmarReservacionCommand	reserva_id, datos_huesped, garantia_pago	ConfirmarReservacionHandler (Colaboradores: ReservaRepository, HabitacionRepository, UnitOfWork, EventBus)	Confirmación de la reserva, identificador único y estado actualizado.	Cambia el estado del sistema, aplica reglas de negocio estrictas (RN-06), protege el inventario y emite un evento de dominio (ReservaConfirmada).
Command	RegistrarCheckInCommand	reserva_id, datos_identidad, datos_fiscales (cuando corresponda emitir factura)	RegistrarCheckInHandler (Colaboradores: ReservaRepository, FolioRepository, UnitOfWork)	Registro del check-in y apertura del folio de estancia.	Modifica el estado de la reserva, asigna la habitación física y abre un nuevo folio activo aplicando validaciones obligatorias de identidad.
Query	ConsultarDisponibilidadQuery	rango_fechas, tipo_habitacion, canal_venta	ConsultarDisponibilidadHandler (Colaboradores: HabitacionRepository, TariffCalculationService)	List<HabitacionDisponibleDTO> con habitaciones disponibles y tarifas cotizadas.	**Solo lectura:** No altera el estado del sistema ni requiere transacciones de escritura; optimiza la presentación rápida de opciones para el cliente.
Query	ObtenerEstadoFolioQuery	folio_id	ObtenerEstadoFolioHandler (Colaboradores: FolioRepository)	FolioSummaryDTO con cargos, consumos y saldo pendiente.	**Solo lectura:** Permite a Recepción consultar el estado de cuenta actual de forma eficiente sin cargar ni bloquear los agregados del dominio de escritura.

Etapa 7 Revisar la propuesta con SOLID

Principio	Pregunta de revision	Evidencia que deben buscar	Impacto	Decisión confirmada
S	¿Cada clase o servicio tiene una razon principal para cambiar?	ReservacionAppService y RecepcionAppService coordinan los casos de uso, mientras que los agregados Reserva y EstanciaFolio protegen las reglas de negocio.	Evita servicios monolíticos (Fat Services), facilitando la mantenibilidad, lectura y pruebas unitarias independientes.	Se mantiene la separación entre coordinación de los casos de uso y lógica de negocio, evitando servicios con demasiadas responsabilidades.

Cisco Confidential

Principio	Pregunta de revision	Evidencia que deben buscar	Impacto	Decisión confirmada
O	¿Pueden agregar una variante de regla sin editar una cadena creciente de condiciones?	TariffCalculationService centraliza el cálculo de tarifas, lo que permite organizar las reglas por temporada, canal de venta o tipo de cliente.	Permite incorporar nuevas reglas de temporada o canales en el futuro sin modificar las estructuras base de las entidades de dominio.	Se propone mantener el cálculo de tarifas en un componente independiente y diseñarlo para facilitar la incorporación de nuevas reglas con cambios mínimos en el código existente.
L	¿Las implementaciones pueden sustituirse sin cambiar el comportamiento esperado?	ReservaRepository, HabitacionRepository y FolioRepository definen contratos que pueden tener implementaciones distintas, como repositorios con ORM o adaptadores en memoria para pruebas.	Garantiza que cualquier implementación concreta de infraestructura pueda intercambiarse libremente sin alterar el comportamiento de los servicios de aplicación.	Se confirma el uso de interfaces, estableciendo que todas sus implementaciones deben respetar los contratos y resultados esperados por los servicios de aplicación.
I	¿Cada consumidor depende solo de las operaciones que utiliza?	Los repositorios se separan según las responsabilidades de reservas, habitaciones y folios, en lugar de concentrar todas las operaciones en una única interfaz.	Los servicios de aplicación consumen únicamente los métodos estrictamente necesarios para su operación, evitando acoplamientos innecesarios con datos ajenos.	Se mantiene la separación de los repositorios para que cada servicio dependa solo de las operaciones que necesita.
D	¿La aplicacion y el dominio dependen de abstracciones?	Los servicios de aplicación dependen de abstracciones como los repositorios y UnitOfWork, mientras que las implementaciones de persistencia pertenecen a infraestructura.	Desacopla por completo el núcleo del negocio de la tecnología de infraestructura, protegiéndolo frente a cambios de frameworks o motores de persistencia.	Se mantiene la dependencia hacia abstracciones para reducir el acoplamiento con las bases de datos, los ORM y otras tecnologías de infraestructura.

Etapa 8 Representar y documentar la arquitectura

Diagrama de clases

Describe la estructura general del software dividida en sus capas de Aplicación, Dominio, Interfaces e Infraestructura. Muestra la composición interna de los Agregados (Reserva y EstanciaFolio), sus Value Objects, Entidades y Servicios de Dominio, así como el desacoplamiento de la base de datos mediante contratos de repositorios y el patrón Unit of Work.

Cisco Confidential

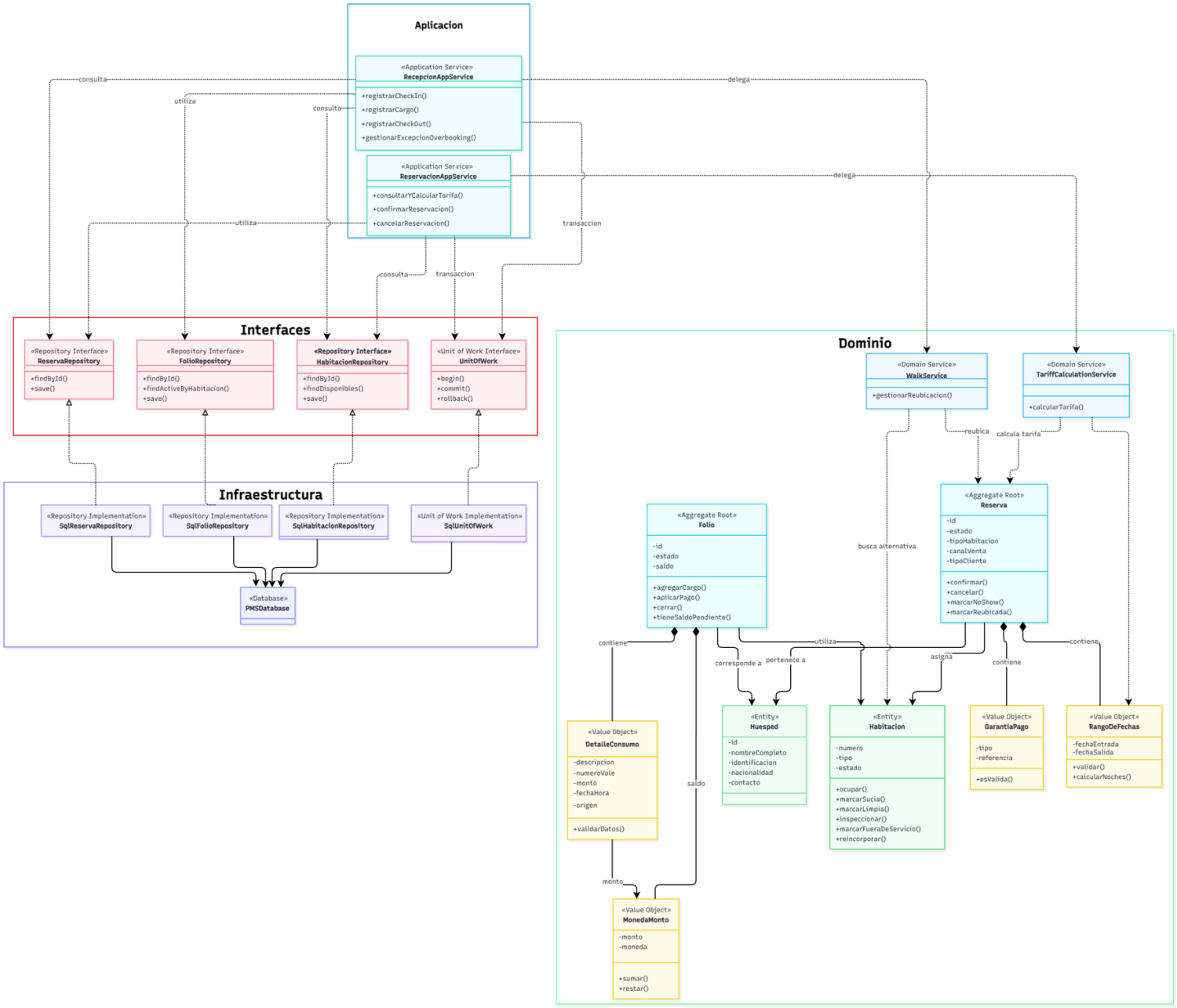


Figura 2: Diagrama de clases general del PMS "Hotel Costa Azul", que ilustra la arquitectura por capas, agregados de dominio, Value Objects y la separación entre interfaces de persistencia e infraestructura.

Diagramas de secuencia

Modela el comportamiento y el intercambio de mensajes en el tiempo ante dos escenarios clave del negocio: el flujo principal de confirmación y persistencia de reservaciones (Secuencia 01), y la atención excepcional de sobreventa o overbooking a través del procedimiento de Walk (Secuencia 02).

Cisco Confidential

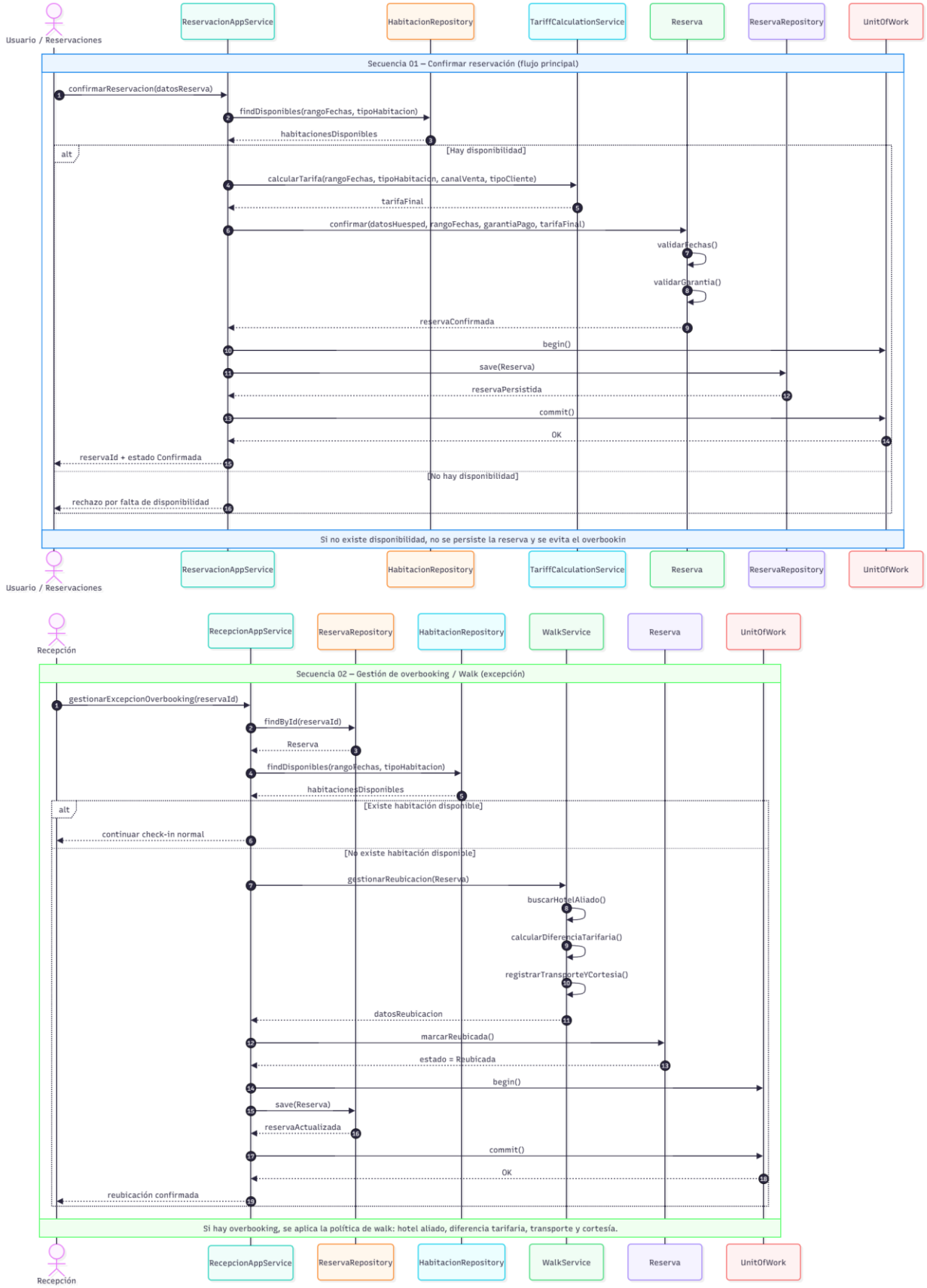


Figura 3: Diagramas de secuencia del PMS "Hotel Costa Azul": Confirmación de reservación (Flujo principal 01) y Gestión de excepción por overbooking / Walk (Flujo de excepción 02).

Cisco Confidential

Architecture Decision Records

Elaboren al menos tres ADR. Cada ADR documentará una decisión que tenga consecuencias para el diseño. Como mínimo, registren la selección del patrón de lógica de negocio y dos decisiones adicionales tomadas durante la actividad.

Campo del ADR	Contenido que debe redactar el equipo
Titulo	Nombre breve de la decision.
Estado	Propuesta, aceptada, reemplazada o descartada.
Contexto	Problema, restricciones y evidencia relevante de U2.
Alternativas	Opciones consideradas y sus consecuencias.
Decision	Opcion elegida y razon concreta.
Consecuencias	Beneficios, costos, riesgos y trabajo posterior.
Trazabilidad	IDs de necesidades, reglas, historias o escenarios relacionados.

ADR elaborados

ADR	Título	Estado	Trazabilidad
ADR-001	Selección del patrón de lógica de negocio: Domain Model	Aceptada	RN-01 a RN-13; US-06, US-13, US-18, US-19, US-23; CU-01 a CU-05
ADR-002	Reserva y Folio como Aggregate Roots; Habitación referenciada por ID	Aceptada	RN-03, RN-04, RN-05, RN-06, RN-07, RN-08, RN-10, RN-12, RN-13; US-02, US-04, US-05, US-06, US-07, US-10, US-11, US-13, US-25; CU-02, CU-03, CU-04
ADR-003	Domain Services y bounded contexts: TariffCalculationService y WalkService	Aceptada	RN-06, RN-13; US-02, US-03, US-23; CU-01, CU-05
ADR-004	Service Layer delgada, Repositories por agregado y Unit of Work	Aceptada	RN-06, RN-12; US-07, US-10, US-13; CU-01 a CU-05
ADR-005	CQRS básico con Commands, Queries y DTOs de lectura	Aceptada	RN-06, RN-12; US-02, US-03, US-07, US-13, US-25; CU-01, CU-02, CU-03, CU-04
ADR-006	Autorización gerencial dividida entre aplicación y dominio	Aceptada	RN-08, RN-10; US-18, US-19, US-23

Matriz de Trazabilidad

La matriz de trazabilidad relaciona cada necesidad del BRD con su regla de negocio, sus historias y criterios, el caso de uso, los componentes del diseño, la decisión (ADR) y la tarea del BPMN donde ocurre. La versión completa, con la prueba que verifica cada regla y las necesidades que se construirán en el proyecto final

Necesidad	Regla de negocio	US / CA	Caso de uso	Componentes del diseño	ADR / BPMN
Cálculo de tarifas base y dinámicas	RN-01, RN-13	US-02, US-03	CU-01	TariffCalculationService, RangoDeFechas, ReservacionAppService	ADR-001, ADR-003 / Tarea: Cotizar Estancia

Cisco Confidential

Verificación y bloqueo de disponibilidad	RN-03, RN-04	US-04, US-05	CU-01	Reserva, HabitacionRepository, ReservacionAppService	ADR-002, ADR-004 / Tarea: Validar Inventario
Gestión de garantías y penalizaciones por cancelación	RN-04, RN-05	US-06, US-25	CU-02	Reserva, GarantiaPago, ReservacionAppService	ADR-002, ADR-005 / Tarea: Cancelar Reserva
Control de datos fiscales obligatorios en check-in	RN-07, RN-12	US-07, US-10	CU-03	EstanciaFolio (Folio), Huesped, RecepcionAppService	ADR-002, ADR-004 / Tarea: Registrar Check-In
Registro de consumos y control de saldos en folios	RN-11, RN-12	US-11, US-13	CU-04	Folio, DetalleConsumo, MonedaMonto, RecepcionAppService	ADR-002, ADR-005 / Tarea: Registrar Cargo / Check-Out
Autorización gerencial para ajustes mayores a USD 50	RN-08, RN-10	US-18, US-19	CU-04	Folio, RecepcionAppService	ADR-006 / Tarea: Aplicar Ajuste Contable
Manejo de overbooking y reubicación (Walk)	RN-06	US-23	CU-05	WalkService, Reserva, RecepcionAppService	ADR-002, ADR-003 / Tarea: Gestionar Excepción Overbooking

Herramientas permitidas

Pueden utilizar draw.io, PlantUML, Mermaid u otra herramienta que permita entregar diagramas legibles. Los archivos fuente deben incluirse en el repositorio; no entreguen únicamente capturas de pantalla.

Vibe Coding con IA

Usarán IA para convertir descripciones del negocio en alternativas de arquitectura, generar estructura inicial, revisar código y producir borradores de diagramas. El equipo sigue siendo responsable de la arquitectura y debe verificar cada resultado contra los documentos de la Unidad 2.

Bitacora obligatoria de IA

Herramienta: Gemini (Google) para el análisis de contexto, modelado de dominio inicial y generación de estructuras; Claude (Anthropic) para la validación, refinamiento estricto de reglas de negocio, revisión SOLID y confirmación de la arquitectura. Las iteraciones se ordenan cronológicamente conforme al avance de las etapas del proyecto. Al inicio de cada prompt se indica entre paréntesis qué integrante lo escribió (Ivan Padrón, María Rodarte o Haziel Bernal), registrando también las preguntas o ajustes de seguimiento realizados sobre la respuesta.

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
1	**Etapa 1:** Preparar el caso.	*(María Rodarte)* Con base en el alcance del Hotel Costa Azul (CU-01 a CU-05, US-02 a US-25), genera la lista de casos de uso estructurada con actores y resultados esperados.	Mapeó correctamente los 5 casos de uso clave vinculando actores, reglas y resultados esperados.	Omitió la regla estricta de bloqueo de datos fiscales en el check-in con factura (US-07).	**Corregido:** Se especificó explícitamente el bloqueo de emisión sin datos fiscales en CU-03.

Cisco Confidential

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
2	**Etapa 2:** Selección de Patrón.	*(María Rodarte)* Con base en [las reglas RN-01 a RN-13, cambios en políticas de cancelación y casos de uso CU-01 a CU-05 del Hotel Costa Azul], compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.	Proporcionó argumentos sólidos analizando la rica interacción entre entidades (Reserva, Folio, Habitación) y la protección de invariantes frente a cambios de políticas.	Sugería Transaction Script para cobros y penalizaciones por considerarlo 'más rápido de programar'.	**Rechazado:** Se eligió **Domain Model** para evitar duplicidad de lógica de penalización (RN-04, RN-05) en múltiples scripts.
3	**Etapa 3:** Mapa de Dominio (Teoría DDD).	*(Ángel Haziel) Revisa este glosario y estas reglas [RN-01 a RN-13, US-01 a US-25 del Hotel Costa Azul]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana.*	Identificó los Agregados Reserva y EstanciaFolio, definiendo RangoDeFechas, MonedaMonto y DetalleConsumo como Value Objects con evidencia del BRD.	Propuso que la entidad Habitación fuera un Aggregate Root independiente desde la primera fase.	**Ajustado:** Se validó mantener Habitación como entidad referenciada por ID para acotar el alcance transaccional.
4	**Etapa 3:** Domain Services y Bounded Contexts.	*Ángel Haziel)* Analiza las dudas del mapa de dominio del Hotel Costa Azul: ¿Dónde deben residir WalkService (overbooking) y TariffCalculationService?	Propuso ubicar TariffCalculationService como un Domain Service stateless desacoplado de la entidad Reserva.	Colocó WalkService dentro del Bounded Context de Reservaciones, ignorando el manejo físico de la estancia.	**Rechazado:** Se reubicó WalkService en el Bounded Context de Estancia por requerir folios y cortesías físicas.
5	**Etapa 4:** Service Layer.	*(María Rodarte) A partir de estos casos de uso [CU-01 a CU-05 y reglas RN-01 a RN-13 del Hotel Costa Azul], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.*	Definió ReservacionAppService y RecepcionAppService con métodos orientados al negocio (confirmarReservacion, registrarCheckOut).	La IA incluyó lógica de cálculo de recargos por late check-out directamente dentro del método de aplicación.	**Corregido**: Se movió la regla del recargo al Agregado EstanciaFolio, dejando el servicio de aplicación como estricto orquestador.
6	**Etapa 5 & 6:** Repositorios y CQRS.	*(María Rodarte)* A partir de los casos de uso CU-01 y CU-04 del Hotel Costa Azul, propón interfaces de Repository, UnitOfWork y la separación CQRS mediante Commands, Queries, Handlers y DTOs.	Generó los DTOs de lectura HabitacionDisponibleDTO y FolioSummaryDTO, desacoplando la consulta rápida de la escritura.	Intentó crear Handlers y modelos CQRS independientes para todas las consultas internas de apoyo del dominio (overengineering).	**Ajustado:** Se justificó no aplicar CQRS a operaciones auxiliares de grano fino para no sobrecomplicar la arquitectura.

Cisco Confidential

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
7	**Etapa 7:** Auditoría SOLID.	*(Ivan Padrón)* Analiza este diseño [servicios de aplicación, agregados y repositorios del Hotel Costa Azul] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida.	Identificó áreas de mejora en Interface Segregation (I) al separar repositorios y en Dependency Inversion (D) al usar interfaces abstractas.	Afirmó que la regla de autorización de ajustes de >$50 USD pertenecía 100% a la capa de Aplicación/Seguridad.	**Corregido:** Se dividió la responsabilidad (Aplicación valida el rol del usuario, el Dominio valida la invariante del límite de $50 USD).
8	**Etapa 8 & 9:** Diagramación y Validación.	*(Ivan Padrón)* Convierte esta arquitectura validada [agregados, servicios de aplicación, repositorios del Hotel Costa Azul] en un borrador PlantUML. No agregues clases, reglas o relaciones que no aparezcan en la descripción.	Generó la estructura en sintaxis PlantUML respetando las clases y relaciones definidas en las etapas anteriores.	Agregó relaciones directas no especificadas entre Huésped y Habitación que violaban los límites de agregados.	**Unificado:** Se corrigió el código PlantUML eliminando las relaciones no especificadas para mantener estricta consistencia con DDD.

Etapa 9 Validar la propuesta

Antes de entregar, recorran cada caso de uso sobre los diagramas y el diseño. El equipo debe poder explicar qué componente recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se persisten los cambios y qué resultado se devuelve.

Validacion	Pregunta	Resultado del equipo
Trazabilidad	¿Cada componente importante responde a una necesidad, regla o caso de uso?	**Sí.** Cada componente estructurado (TariffCalculationService, WalkService, Reserva, EstanciaFolio) está explícitamente vinculado en la matriz de trazabilidad con las reglas de negocio (RN-01 a RN-13), historias de usuario y casos de uso (CU-01 a CU-05) correspondientes.
Lenguaje	¿Los mismos terminos significan lo mismo en BRD, UML, codigo y ADR?	**Sí.** Se mantiene un rigor estricto con el Lenguaje Ubicuo en todo el glosario, los diagramas UML, los registros ADR y las especificaciones de clases (ej. términos como Aggregate Root, Folio, BAR, Walk).
Responsabilidad	¿Las reglas permanecen en el dominio o en el patron elegido con una justificacion?	**Sí.** Las reglas de negocio, validaciones y restricciones operativas residen de forma exclusiva en los agregados (Reserva, EstanciaFolio) y servicios de dominio, asegurando que la capa de aplicación funcione únicamente como orquestador delgado.

Cisco Confidential

Validacion	Pregunta	Resultado del equipo
Consistencia	¿Los diagramas de clase y secuencia representan la misma arquitectura?	**Sí.** Tanto el diagrama de clases general como los diagramas de secuencia (flujo principal y de excepción) comparten exactamente los mismos agregados, repositorios, servicios y patrones arquitectónicos bajo Clean Architecture.
Persistencia	¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura?	**Sí.** El dominio está totalmente aislado mediante la utilización de interfaces de repositorio orientadas al negocio y el patrón Unit Of Work, delegando los detalles de base de datos a la capa de infraestructura.
Transaccion	¿El limite de la operacion y su manejo de error estan definidos?	**Sí.** Los límites transaccionales de los casos de uso están delimitados claramente mediante el patrón UnitOfWork (begin, commit, rollback), asegurando la atomicidad y el manejo estructurado de errores con mensajes de negocio.
IA	¿Cada aporte de IA fue revisado y su uso esta registrado?	**Sí.** Todos los borradores, estructuras y propuestas generadas por la IA fueron auditados, corregidos (como el ajuste de límites de agregados y responsabilidades) y documentados formalmente en la bitácora obligatoria.

CONCLUSIÓN

Como equipo, logramos transformar el análisis de negocio del **"Hotel Costa Azul"** en una propuesta de arquitectura robusta basada en *Domain Model* y *Clean Architecture*.

Mediante el diseño de agregados (`Reserva`, `EstanciaFolio`) y servicios especializados, aseguramos la protección estricta de las reglas del negocio, el control de inventario y la integridad transaccional. La separación mediante repositorios abstractos, el patrón *Unit of Work* y CQRS básico nos otorgan una base desacoplada y flexible. Gracias a la trazabilidad y la revisión SOLID, el equipo entrega una arquitectura sólida y lista para la fase de implementación del sistema.

Link de GitHub: https://github.com/xiro07/Equipo-9.git

Cisco Confidential